

Gaze-Controlled Mouse Cursor Using Wearable Eye Camera and Neural Networks

Course: Neural Networks – Deep Learning

Author: [Aimilios Mavrothalassitis AEM:10983] **Date:** [07/01/2026]

All code will be available in my github: <https://github.com/EmiliosBlacksea?tab=repositories>

Abstract

This project implements a low-cost gaze-tracking system that controls a laptop mouse cursor based on where the user looks. The hardware consists of a 3D-printed pair of glasses with a high-FPS mini camera aimed at the left eye, combined with the laptop webcam to capture head/glasses context. A custom dataset was created by displaying a black dot at random screen locations on a 1920x1080 display, waiting about 1 second for fixation, and capturing synchronized images. Each labeled sample contains a composite input image (left-eye image plus a rotated laptop-camera crop placed next to it) and a target label (dot_x, dot_y). Additionally, about 40k unlabeled samples were collected during natural viewing at about 0.5 second intervals to support representation learning. Models compared include (1) ResNet18 end-to-end regression predicting gaze coordinates, (2) ResNet18 feature extraction followed by SVR, and (3) a fully unsupervised autoencoder trained to reconstruct 512-dimensional ResNet feature embeddings (512-124-512) using MSE. Evaluation uses Euclidean pixel error. Best supervised results reach about 19 px median and 27 px mean test error. A real-time demo was implemented using the regression model; the SVR approach was not used live due to time constraints and unstable behavior in the live pipeline.

1. Introduction

Hands-free pointing can improve accessibility and ergonomics and enables interaction when a mouse or trackpad is inconvenient. Eye gaze is a natural signal for pointing tasks, but commercial gaze trackers are often costly or require specialized sensors. This project explores a practical approach using inexpensive camera hardware integrated into 3D-printed glasses and neural networks to estimate gaze location on a laptop screen, including dataset creation, model training, evaluation, and real-time cursor control.

2. Problem Statement

Given synchronized camera observations (a close-up grayscale image of the left eye and a grayscale laptop-webcam view of the glasses/head region), predict the screen coordinates (x, y) of the user's gaze target and map that prediction to the laptop cursor position. Coordinates are defined in pixels with top-left origin on a 1920x1080 screen.

3. System Overview

3.1 Hardware

Eye camera module: GC0308 mini camera module (0.3 MP, about 120 FPS) mounted on the glasses and aimed at the left eye.

Label leakage avoidance: A screen-facing camera was initially considered, but since ground-truth labels are created by rendering a visible dot on the screen, a screen-facing camera would observe the label directly and leak ground truth into the inputs. The final supervised system therefore uses only the left-eye camera plus the laptop webcam.

3.2 Mechanical Design

A pair of glasses and camera mounts were 3D printed, including bases that hold, align, and focus the eye camera. A green sticker marker was placed on the glasses to assist in locating the glasses region in the laptop webcam image. The prototype is documented in Figures 1-2.



[Figure 1: glasses_front_view - Front view of the 3D-printed glasses prototype with mounted eye camera.]



[Figure 2: glasses_side_view - Left view showing camera alignment and mounting geometry.]

3.3 Data Capture Software

Data was recorded on the laptop using a Python script that connects to both cameras and saves synchronized frames. For labeled collection, a fixation delay of about 1 second was used before saving the sample.

4. Dataset

4.1 Sample Format (Composite Input)

Each sample is stored as one composite grayscale image. The left-eye camera image is used in landscape orientation. The laptop webcam image is rotated and placed next to the eye image. The combined composite is resized to 224x224 for the network input.



4.2 Labeled Data Collection

A black dot is displayed at a random location on the 1920x1080 screen. The system waits about 1

second for fixation, then captures synchronized frames and stores a composite image and its label (dot_x, dot_y) in a CSV/Excel file.

Label file columns: filename, dot_x, dot_y, split.

4.3 Unlabeled Data Collection

Unlabeled samples were captured during natural viewing at about 0.5 second intervals while wearing the glasses. These samples were used for unsupervised representation learning.

4.4 Dataset Size and Users

The dataset contains about 40,000 labeled samples and about 40,000 unlabeled samples from 4 users (author, mother, brother, grandmother). Approximately 30k labeled samples and essentially all unlabeled samples were collected from the author; about 5-7k labeled samples were collected from the mother; the remainder were collected from the brother and grandmother.

4.5 Cleaning and Marker Failures

The laptop webcam processing relies on detecting a green sticker marker on the glasses. When the marker was not detected, the crop could fall back to an incorrect/random region; these invalid samples were manually discarded during dataset cleaning.

4.6 Train/Test Split-

An 85% / 15% train/test split was used (random assignment stored in the label file).

4.7 Target Normalization

Labels are stored in pixels but normalized during training to $[0,1]$: $x_norm = dot_x/1920$, $y_norm = dot_y/1080$. Models predict normalized coordinates which are denormalized back to pixels for evaluation and cursor control.

5. Preprocessing and Augmentation

All inputs were converted to grayscale. Eye-camera orientation corrections (flips and rotations) were applied to align the eye view. For ResNet18, the single grayscale channel was repeated 3 times to form a 3-channel input. The composite image was resized to 224x224.

6. Modeling Approaches

6.1 ResNet18 End-to-End Regression

A ResNet18 backbone was used with a 2-neuron regression head predicting (x_norm, y_norm) . The model was trained end-to-end on labeled data.

6.2 Evaluation Metric

Performance is reported as Euclidean pixel error after denormalizing predictions back to pixel units:

```
px_error = sqrt((x_pred - x_true)^2 + (y_pred - y_true)^2)
```

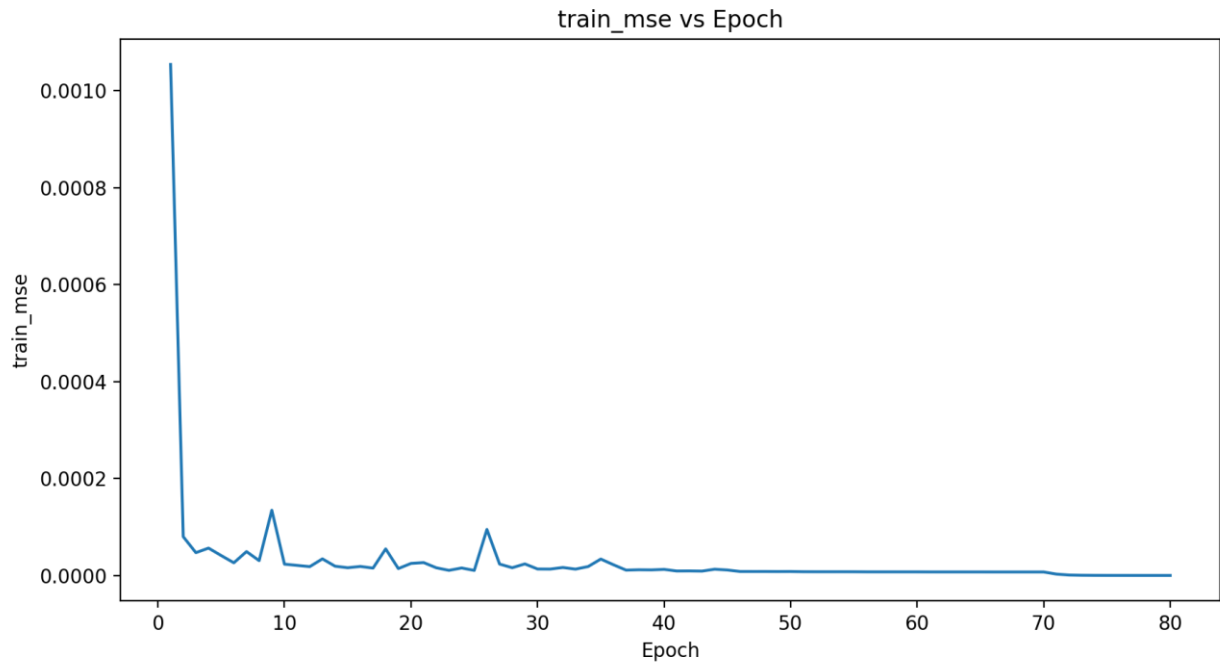
6.3 ResNet18 Feature Extractor + SVR

ResNet18 was used as a feature extractor to produce a 512-dimensional embedding after global average pooling. An SVR model (RBF kernel) was trained to regress gaze coordinates. Hyperparameters (C, epsilon, gamma) were tuned on a subset and then refit on the full training set.

Best SVR parameters: kernel=rbf, C=100, epsilon=0.005, gamma=1e-05.

6.4 Unsupervised Autoencoder on ResNet Features (Project 3)

A fully unsupervised autoencoder was trained in feature space. The input is the 512-dimensional ResNet embedding and the autoencoder reconstructs the same vector using MSE loss. The architecture is 512-124-512. Because this stage does not require labels, it can use both labeled and unlabeled samples. The intended next step is to train a gaze regressor on the encoder representation and compare against supervised baselines.



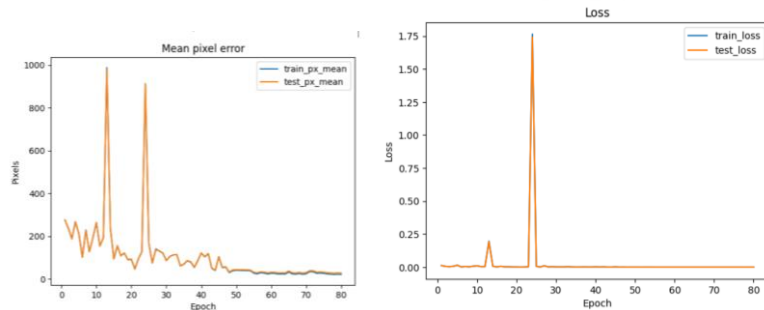
7. Training Notes

Regression training used a warm-up stage followed by unfreezing the ResNet backbone. Learning rate started at 0.003 and decayed in stages. Training was generally stable but exhibited rare spikes in pixel error that later recovered. SVR training included a parameter sweep on a 20k subset (fit=16k, val=4k), followed by final training on the full extracted feature set.

8. Results

8.1 ResNet18 Regression

Near the end of training, the regression model achieved approximately a test mean error of 27-28 px and a test median error of 19-20 px. Example late-epoch performance includes test mean 27.3 px / median 19.1 px (epoch 78) and test mean 27.5 px / median 19.0 px (epoch 80). A small number of epochs exhibited extreme spikes (around 900-1000 px) before the model recovered, suggesting episodic instability (e.g., outlier batches or preprocessing/crop failures).



[Figure 3: loss + px accuracy (mean) for ResNet18 + FC regression head.]

8.2 ResNet18 Features + SVR

Final SVR performance: train mean 19.95 px / median 15.14 px; test mean 26.76 px / median 18.64 px. Test tail errors were p90 44.93 px, p95 62.78 px, and p99 174.15 px. Although SVR slightly improved offline metrics, it was not used in the live demo due to time constraints and unstable real-time behavior; a possible cause is mismatch between training and live input conditions or overfitting of the SVR mapping.

8.3 Autoencoder-Based Experiments (Project 3)

Two AE regimes were evaluated: (1) before unfreezing the ResNet feature extractor and (2) after unfreezing at about epoch 70. Before unfreezing (epoch <70), AE-based regression achieved performance comparable to the supervised baseline but did not exceed it. After unfreezing (epoch ≥70), downstream gaze prediction performance degraded significantly, indicating embedding drift/instability under the reconstruction objective.

8.4 Interpreting Pixel Error on 1920x1080

On a 1920x1080 screen, the SVR test median error of 18.64 px corresponds to about 0.97% of screen width, and the test mean error of 26.76 px corresponds to about 1.39% of screen width. Relative to the screen diagonal (~2202.9 px), median error is about 0.85% and mean error about 1.21%.

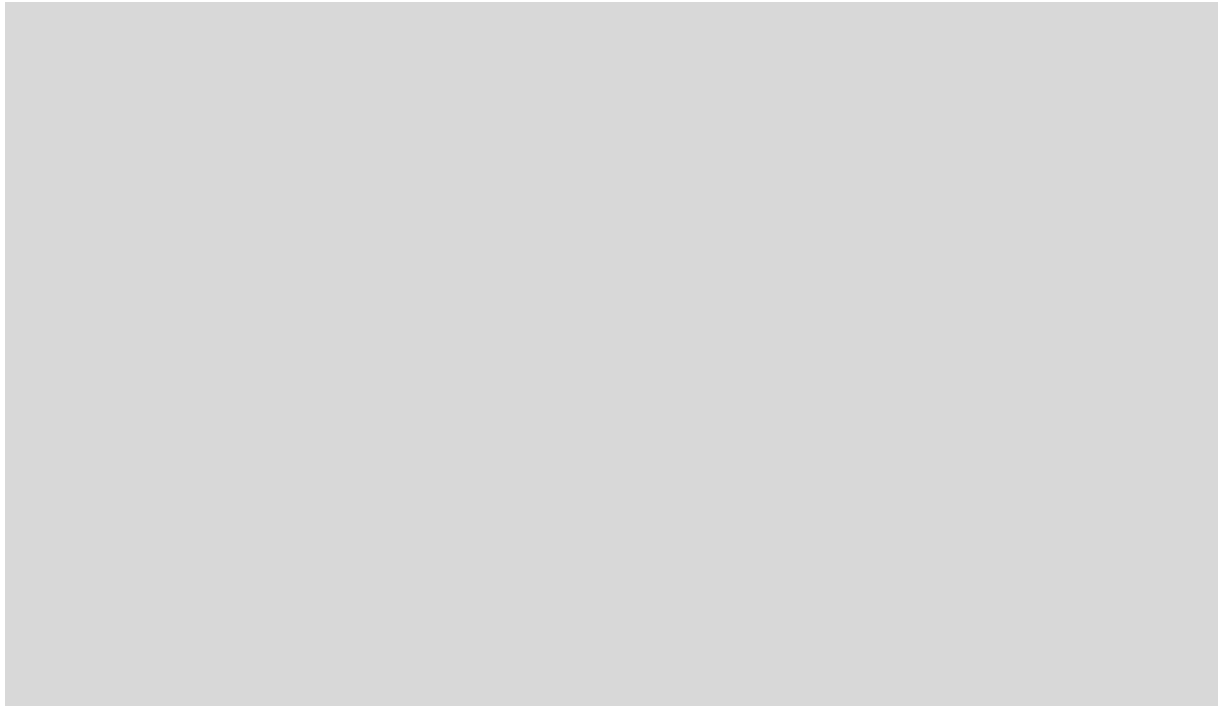
8.5 Summary Comparison

Method	Test Mean (px)	Test Median (px)	Notes
ResNet18 end-to-end regression	≈ 27-28	≈ 19-20	Used in live demo. Rare unstable epochs.
ResNet18 features + SVR (RBF)	26.76	18.64	Slightly better offline. Live pipeline not stabilized.
ResNet18 + AE (feature-space)	Did not improve	Did not improve	Pre-unfreeze similar to baseline; post-unfreeze degraded.

9. Real-Time Cursor Control

A real-time pipeline was implemented: synchronized frames are captured from the eye camera and laptop webcam, combined into the composite input, passed through the regression model, and the raw normalized prediction (x_{norm} , y_{norm}) is mapped directly to screen pixels: $x = x_{\text{norm}} * 1920$

and $y = y_norm * 1080$. The main practical issues observed were reduced accuracy near screen edges, sensitivity to lighting changes, and degradation under large head movements.



[Figure 7: Live demo video (provided separately).]

10. Discussion

Why SVR helps offline: SVR on top of deep embeddings can provide a strong inductive bias for smooth regression while leveraging the representational power of ResNet features. In these experiments, SVR slightly outperformed end-to-end regression on mean and median test error.

Why SVR may fail in real time: Even with low offline error, live performance can degrade due to distribution shift (lighting, head movement), small differences in preprocessing, or sensitivity of the SVR mapping. Time constraints prevented full debugging and stabilization of the live SVR pipeline.

Autoencoder outcome: The feature-space autoencoder did not improve best accuracy. Unfreezing the ResNet backbone during AE training degraded downstream regression, likely due to embedding drift where the backbone optimizes reconstruction rather than preserving gaze-relevant geometry.

11. Limitations

- Random splitting may overestimate generalization; stronger evaluation would use user-held-out or session-held-out splits.
- Performance depends on stable camera alignment and correct marker-based cropping; failures contribute to tail errors.
- Pixel error does not fully capture usability; jitter and latency affect pointing quality.

12. Future Work

- Complete encoder-to-regression training and compare against regression and SVR baselines.
- Evaluate with user-held-out and session-held-out splits to test generalization under lighting and motion changes.
- Improve real-time stability with smoothing (EMA/moving average/Kalman), outlier rejection, and

blink/bad-frame detection.

- Add a click interaction method (dwell time, blink, or hotkey).

Appendix A: Label File Format

Columns: filename, dot_x, dot_y, split.

Appendix B: SVR Parameters and Tail Errors

Best parameters: kernel=rbf, C=100, epsilon=0.005, gamma=1e-05. Test tails: p90 44.93 px, p95 62.78 px, p99 174.15 px.